

四位乘法器设计

1、实验目的:

学习应用移位相加原理设计四位乘法器。

2、实验原理:

该乘法器是由 4 位加法器构成的以时序方式设计的 4 位乘法器。原理是：乘法通过逐项移位相加来实现相乘。从被乘数的最低位开始，若为 1，则乘数左移后与上一次的和相加；若为 0，左移后以全零相加，直至被乘数的最高位。

3、实验程序:

```
module mul_con(
input clk,
input reset_n,
input mul1_key,
input mul2_key,
output reg [7:0] mul1,
output reg [7:0] mul2
);
reg [7:0] mul1_buf, mul2_buf;
always @ (negedge mul1_key or negedge reset_n)
if(!reset_n)
begin
mul1_buf <= 0;
end
else if (!mul1_key)
mul1_buf <= mul1_buf + 1'b1;
always @ (negedge mul2_key or negedge reset_n)
if(!reset_n)
begin
mul2_buf <= 0;
end
else if (!mul2_key)
mul2_buf <= mul2_buf + 1'b1;
always @(mul1_buf or mul2_buf)
begin
mul1 <= mul1_buf;
mul2 <= mul2_buf;
end
endmodule
```

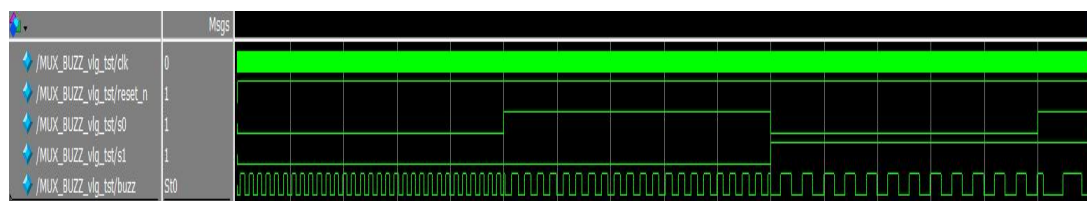
```

module M8B (R,A,B);
    parameter S=4;
    output[2*S:1] R ;
    input[S:1] A,B ;
    reg[2*S:1] R,AT; reg[S:1] BT,CT;
    always @(A,B) begin
        R=0; AT = {{S{1'B0}},A};
        BT = B; CT = S;
        for(CT=S; CT>0; CT=CT-1)
            begin if(BT[1]) R=R+AT;
                AT = AT<<1; //左移 1 位
                BT = BT>>1; //右移 1 位
            end
        end
    end
endmodule

```

这两则使用了 for 循环语句的以移位相加方式实现的 4 位乘法器的设计。对于循环控制变量增值表达式，前者采用增值的方式，后者采用了减值的方式。

4、实验仿真：



5、引脚约束：

#时钟周期约束

```

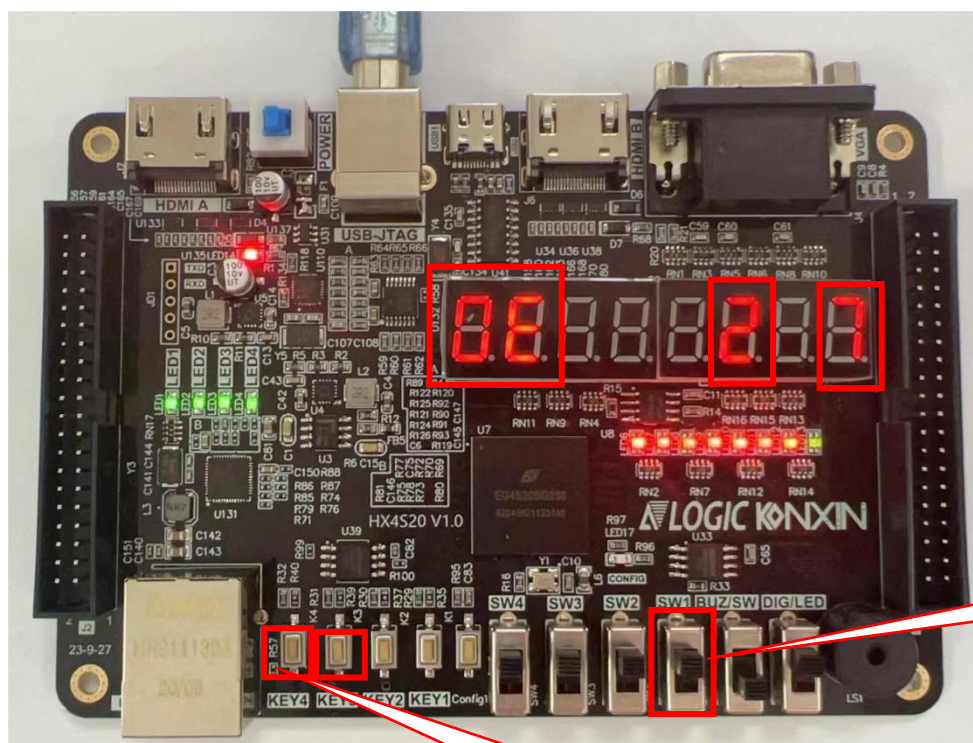
set_pin_assignment { DIG[0] } { LOCATION = C15; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { DIG[1] } { LOCATION = C16; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { DIG[2] } { LOCATION = B6; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { DIG[3] } { LOCATION = A5; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { DIG[4] } { LOCATION = B5; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { DIG[5] } { LOCATION = A8; IOSTANDARD = LVCMOS33;
DRIVESTRENGTH = 8; PULLTYPE = NONE; }

```

```
set_pin_assignment { DIG[6] } { LOCATION = A7; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[0] } { LOCATION = D11; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[1] } { LOCATION = E11; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[2] } { LOCATION = F10; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[3] } { LOCATION = C13; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[4] } { LOCATION = E10; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[5] } { LOCATION = E12; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[6] } { LOCATION = E16; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { SEL[7] } { LOCATION = F16; IOSTANDARD = LVCMOS33;  
DRIVESTRENGTH = 8; PULLTYPE = NONE; }  
set_pin_assignment { clk } { LOCATION = R7; IOSTANDARD = LVCMOS33;  
PULLTYPE = PULLUP; }  
set_pin_assignment { mul1_key } { LOCATION = A2; IOSTANDARD = LVCMOS33;  
PULLTYPE = PULLUP; }  
set_pin_assignment { mul2_key } { LOCATION = B2; IOSTANDARD = LVCMOS33;  
PULLTYPE = PULLUP; }  
set_pin_assignment { reset_n } { LOCATION = C8; IOSTANDARD = LVCMOS33;  
PULLTYPE = PULLUP; }
```

6、下载测试:

将开关 SW6 拨到 DIG 档, 初始左侧两个数码管以及右侧两个数码管显示 0, 点击 KEY3,KEY4 可以分别给两个乘数加 1, 此时左侧两个数码管会对应显示乘数的值, 左侧侧两个数码管显示 16 进制的乘法结果。本设计只能显示两个乘数为 1-15 的值, 若继续点击, 对应乘数的数码管会熄灭, 而右侧显示结果的数码管只会显示低 8 位的值。



SW1 复位

KEY1,KEY2 分别给两个乘数加 1